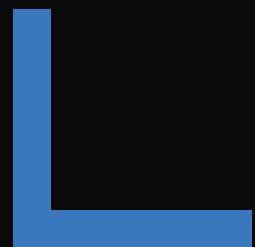




**2.2.1 Técnicas avanzadas de
diseño de casos de prueba**
Creación de casos de prueba
utilizando técnicas específicas



Contenido

01

Introducción y Conceptos Fundamentales

02

Clasificación de Técnicas de Diseño

04

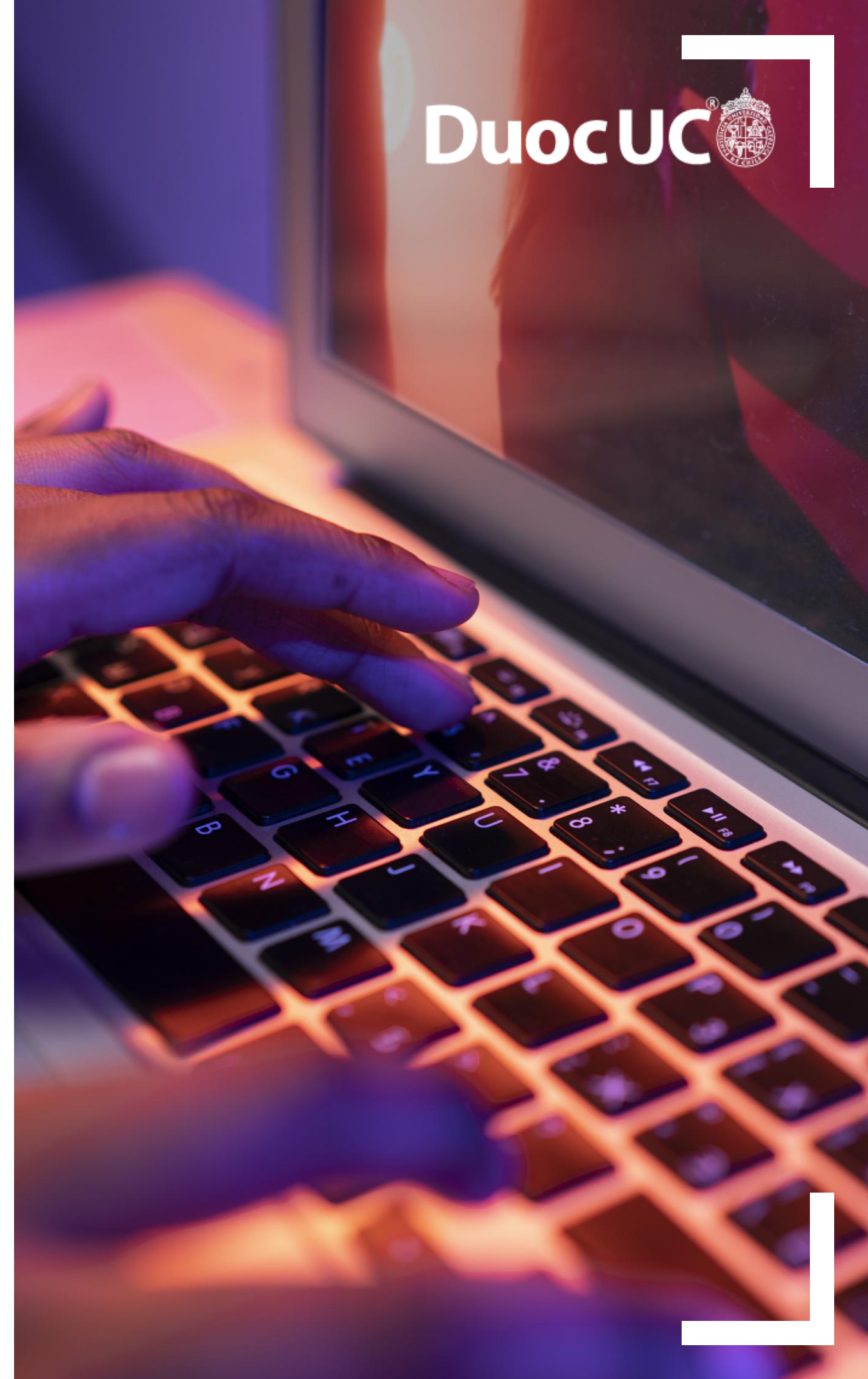
Mejores Prácticas y Cierre

03

Profundización en Caja Blanca y Métricas

04

Creación Práctica y Gestión



02

Técnicas avanzadas de diseño de casos de prueba

Creación de casos de prueba
utilizando técnicas específicas

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
  color: white;  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```

El Desafío del Tester "Detective"

En lugar de empezar con definiciones, vamos a activar el pensamiento crítico de inmediato.

- . **"Si hoy entregáramos un software que 'parece' funcionar bien pero no tiene ni un solo caso de prueba documentado, ¿qué es lo peor que podría pasar el lunes por la mañana?"**.
- . **"¿Qué hace que un caso de prueba sea 'bueno'? ¿Su complejidad o su claridad?"**.

Los casos de prueba no son solo burocracia; según Mc Mullin, son la herramienta principal para reducir significativamente los costos y tiempos en el ciclo de vida del software.

01. Introducción y Conceptos Fundamentales

Fundamentos de los Casos de Prueba

Definición IEEE 610

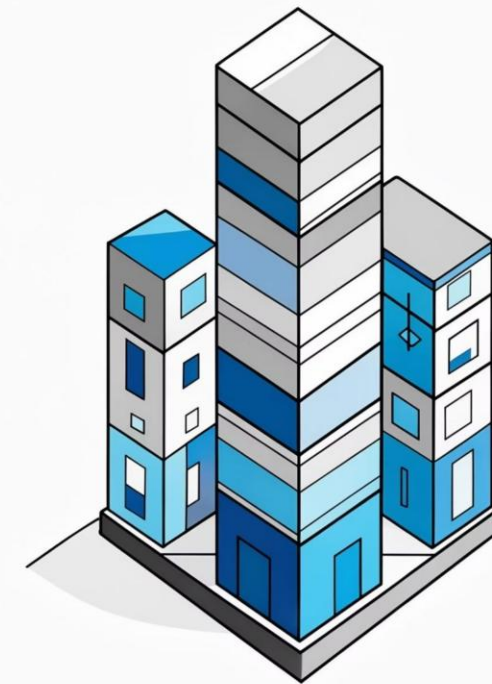
Un caso de prueba es un conjunto estructurado de entradas, condiciones de ejecución y resultados esperados desarrollado para validar un requisito específico del sistema.

Importancia crítica

Los casos de prueba forman la base fundamental para garantizar la calidad del software, detectar errores tempranamente y asegurar que la funcionalidad cumple con las expectativas.

Beneficios clave

Mejoran significativamente la comunicación entre equipos, aumentan la cobertura de pruebas y contribuyen a reducir costos operativos y tiempos de desarrollo.



El papel crítico de los casos de prueba bien diseñados

Guía uniforme de ejecución

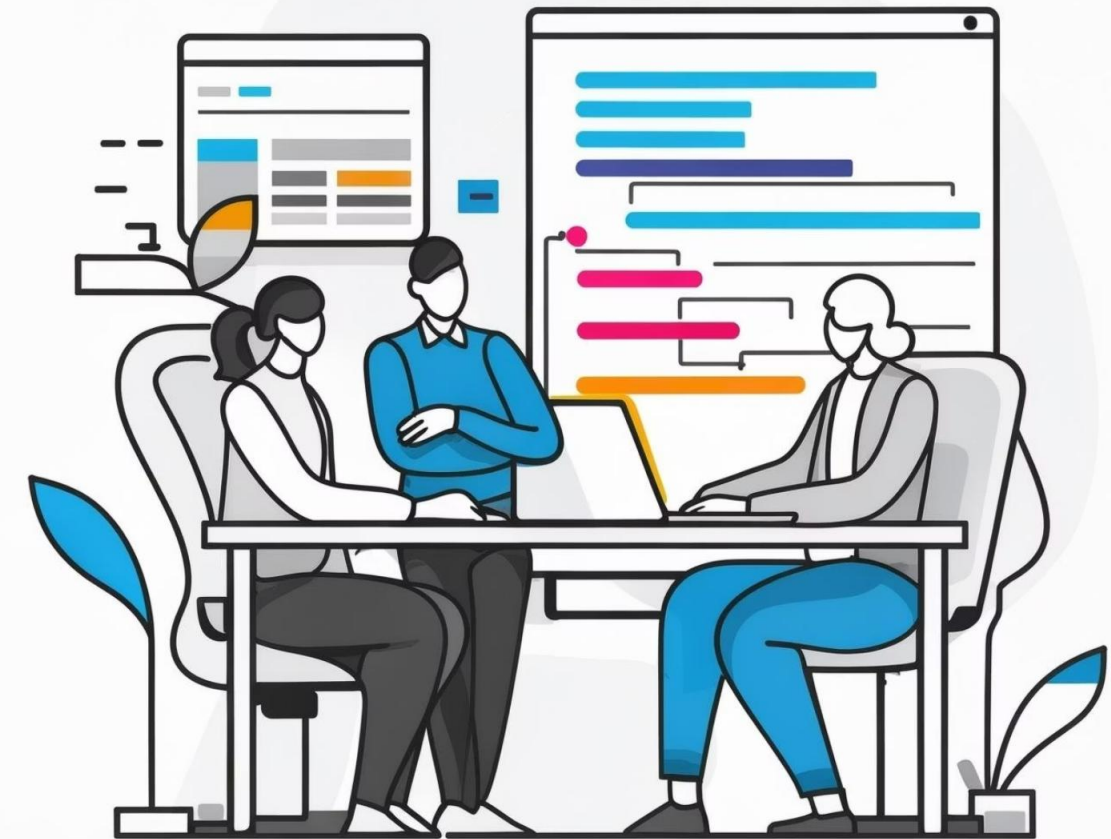
Los casos de prueba bien estructurados proporcionan instrucciones claras y consistentes que aseguran que todos los miembros del equipo ejecuten las pruebas de manera uniforme, eliminando ambigüedades y confusiones.

Colaboración mejorada

Facilitan la comunicación efectiva entre desarrolladores, testers y stakeholders, creando un lenguaje común que todos pueden entender y seguir.

Automatización eficiente

Permiten la automatización sistemática y la reutilización eficiente de pruebas, ahorrando tiempo y recursos valiosos en ciclos de desarrollo futuros.



02: Clasificación de técnicas de diseño

1

Técnicas de caja negra

Se centran exclusivamente en las entradas y salidas del sistema sin necesidad de conocer la implementación interna del código. El tester valida la funcionalidad desde la perspectiva del usuario final.

- Partición de equivalencia
- Análisis de valores límite
- Tablas de decisión

2

Técnicas de caja blanca

Requieren conocimiento profundo de la estructura interna del software, incluyendo código fuente, flujos de control y arquitectura. Permiten validar la lógica interna y optimización del código.

- Pruebas de caminos básicos
- Cobertura de sentencias
- Análisis de flujo de datos

3

Técnicas híbridas

Combinan elementos de ambos enfoques anteriores, proporcionando una estrategia equilibrada que aprovecha las ventajas de caja negra y caja blanca para maximizar la cobertura de pruebas.

- Pruebas basadas en riesgos
- Pruebas exploratorias estructuradas

Técnicas de caja negra: Partición de equivalencia

La partición de equivalencia es una técnica fundamental que optimiza el proceso de prueba al dividir el dominio completo de entrada en clases o particiones equivalentes. Esta estrategia inteligente se basa en la premisa de que si un valor de una clase funciona correctamente, todos los valores de esa clase deberían comportarse de manera similar.

01

Identificar el dominio de entrada

Analizar todas las posibles entradas del sistema y sus características

02

Dividir en clases equivalentes

Agrupar entradas que producen el mismo comportamiento del sistema

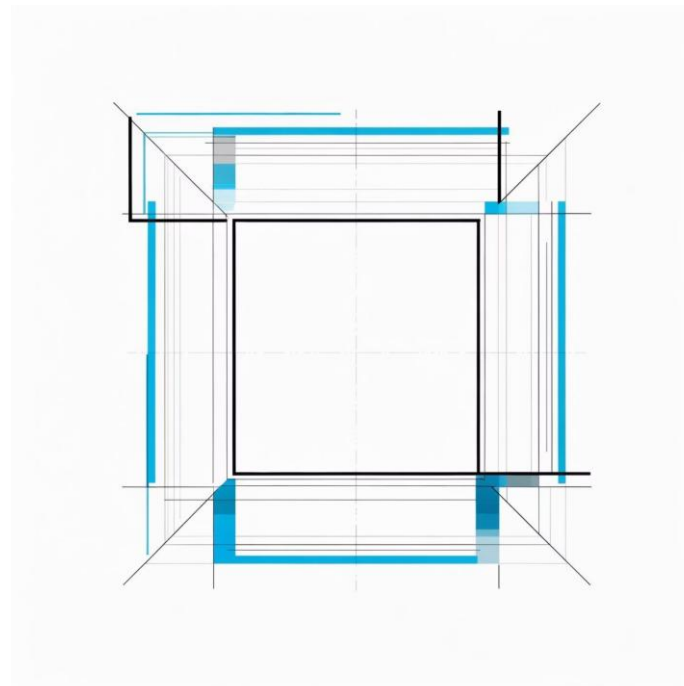
03

Seleccionar valores representativos

Elegir un valor de cada clase para ejecutar las pruebas

- ❏ **Ejemplo práctico:** Para un sistema que acepta valores válidos en rangos 1-10 y 20-30, identificamos tres clases: valores válidos primera mitad (1-10), valores válidos segunda mitad (20-30), y valores inválidos (fuera de ambos rangos). Un solo valor de cada clase es suficiente para la validación.

Técnicas de caja negra: Análisis de valores límite (BVA)



El análisis de valores límite es una técnica poderosa que se enfoca específicamente en probar los valores que se encuentran en los límites o fronteras de las particiones de equivalencia. La experiencia demuestra que la mayoría de los errores ocurren precisamente en estos puntos críticos.

¿Por qué son importantes los límites?

Los errores comunes en comparaciones (como confundir menor que con menor o igual) se manifiestan típicamente en los valores límite. Esta técnica detecta estos problemas de manera efectiva.



Límite inferior

Probar: valor mínimo - 1, valor mínimo, valor mínimo + 1



Valores centrales

Incluir valores representativos del rango medio para completar cobertura



Límite superior

Probar: valor máximo - 1, valor máximo, valor máximo + 1

Ejemplo ilustrativo: Para un rango válido de 1 a 10, los casos de prueba serían: 0 (justo antes), 1 (límite inferior), 2 (dentro del rango), 9 (dentro del rango), 10 (límite superior), y 11 (justo después).

Técnicas de caja negra: Tablas de decisión

Las tablas de decisión son herramientas excepcionalmente útiles cuando se trabaja con funcionalidades que involucran múltiples condiciones combinadas y diferentes resultados según dichas combinaciones. Proporcionan una representación visual clara y sistemática de todas las posibles combinaciones de entradas y sus correspondientes resultados esperados.

Ventajas principales

- Visualización clara de todas las combinaciones posibles de condiciones
- Identificación sistemática de casos de prueba sin omisiones
- Fácil detección de inconsistencias en la lógica de negocio
- Documentación comprensible para stakeholders no técnicos

Componentes de una tabla de decisión

1. **Condiciones:** Las diferentes variables o factores que afectan el comportamiento
2. **Combinaciones:** Todas las posibles combinaciones de valores de las condiciones
3. **Acciones:** Los resultados o comportamientos esperados para cada combinación

Visualización: Partición de equivalencia y valores límite

Clase inválida inferior

Valores menores que el mínimo

Ejemplo: -5, -1, 0

Límite inferior crítico

Transición de inválido a válido

Probar: 0, 1, 2

Clase válida central

Valores dentro del rango aceptable

Ejemplo: 3, 5, 8

Límite superior crítico

Transición de válido a inválido

Probar: 9, 10, 11

Clase inválida superior

Valores mayores que el máximo

Ejemplo: 12, 15, 100

Este diagrama conceptual ilustra cómo las técnicas de partición de equivalencia y análisis de valores límite trabajan conjuntamente para proporcionar una cobertura completa y eficiente del espacio de entrada.

03: Técnicas de caja blanca avanzadas

Las técnicas de caja blanca requieren un conocimiento profundo del código fuente y la estructura interna del software. A diferencia de las técnicas de caja negra, estas permiten validar no solo que el sistema funcione correctamente, sino también que lo haga de manera eficiente y óptima.



Pruebas de caminos básicos

Garantizan que cada camino lógico independiente en el código se ejecute al menos una vez, asegurando cobertura completa de todas las rutas de ejecución posibles.



Pruebas de condiciones

Validan exhaustivamente todas las expresiones condicionales (if, switch, operadores lógicos) para verificar que cada rama del código funcione correctamente en todos los escenarios.



Pruebas de bucles

Se enfocan en validar todos los tipos de ciclos (for, while, do-while) con diferentes números de iteraciones, incluyendo casos extremos como cero iteraciones o muchas iteraciones.

Complejidad ciclomática y diseño de casos de prueba

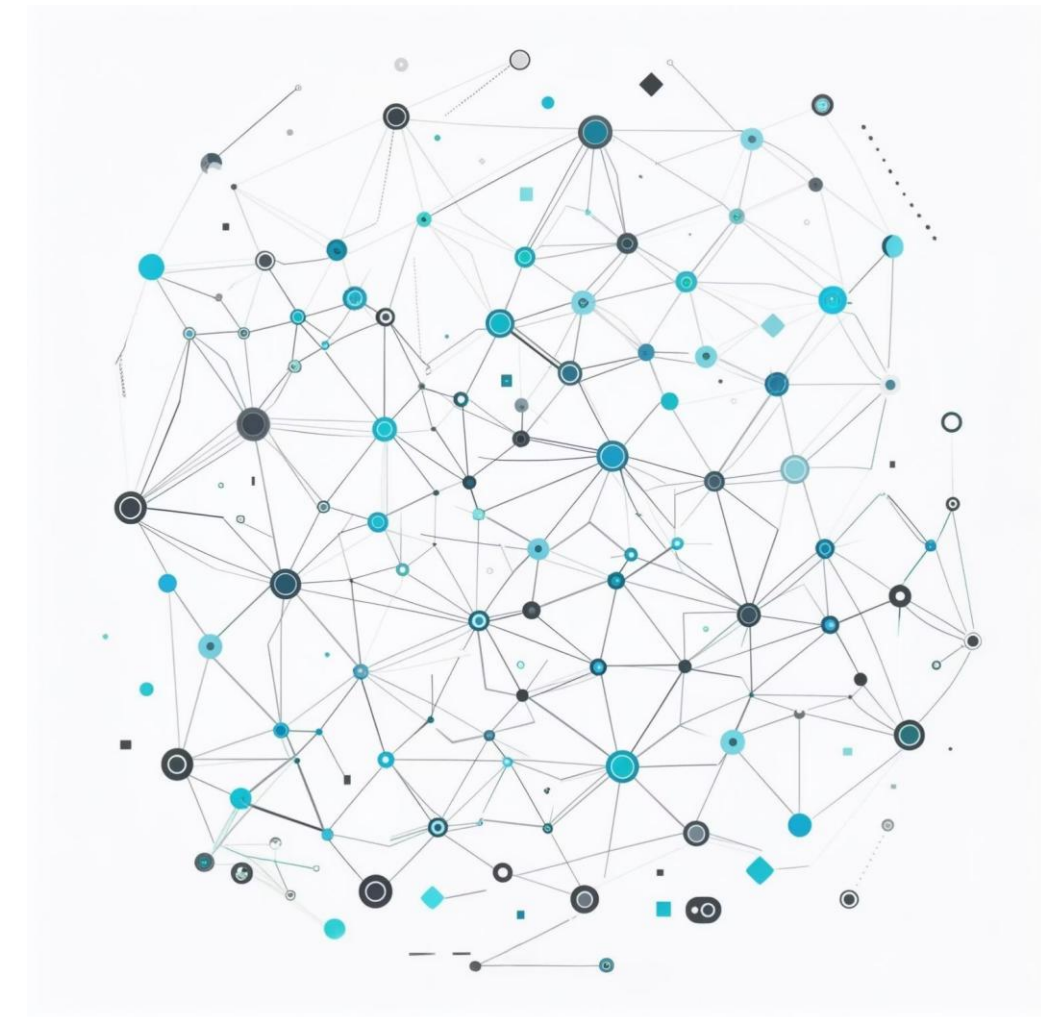
La complejidad ciclomática es una métrica cuantitativa fundamental en ingeniería de software que mide el número de caminos linealmente independientes a través del código fuente de un programa. Esta métrica, desarrollada por Thomas McCabe en 1976, proporciona una guía objetiva para determinar cuántos casos de prueba mínimos se necesitan para lograr una cobertura completa.

Cálculo de la complejidad ciclomática

Se puede calcular mediante la fórmula: $V(G) = E - N + 2P$, donde E representa las aristas del grafo de flujo, N los nodos, y P los componentes conectados.

Interpretación práctica

- $V(G) = 1-10$: Código simple, fácil de probar
- $V(G) = 11-20$: Complejidad moderada
- $V(G) = 21-50$: Código complejo, requiere refactorización
- $V(G) > 50$: Código muy complejo, alto riesgo



❏ **Ejemplo aplicado:** Una función con complejidad ciclomática de 5 requiere un mínimo de 5 casos de prueba diferentes para garantizar que todos los caminos independientes sean ejecutados al menos una vez.

Pruebas de bucles y flujo de datos

Estrategias para pruebas de bucles

Los bucles son estructuras críticas que requieren atención especial durante las pruebas, ya que son fuente común de errores como desbordamientos, condiciones incorrectas de terminación y problemas de rendimiento.

Bucles simples (n iteraciones máximas)

Probar con: 0 iteraciones (omitir bucle), 1 iteración, 2 iteraciones, valor típico (m , donde $m < n$), $n-1$, n , $n+1$ iteraciones

Bucles anidados

Comenzar desde el bucle más interno hacia afuera, probando cada nivel mientras los demás mantienen valores mínimos. Validar todas las combinaciones críticas de iteradores.

Bucles concatenados

Si son independientes, aplicar pruebas de bucles simples a cada uno. Si están relacionados, tratarlos como bucles anidados para validar interacciones.

Análisis de flujo de datos

Verificar el uso correcto de variables a lo largo del flujo del programa, identificando patrones como: definición-uso, definición-redefinición sin uso, uso sin definición previa. Esto previene errores sutiles relacionados con el estado de los datos.

Combinación de técnicas para mayor efectividad

"Ninguna técnica de prueba por sí sola puede garantizar la calidad completa del software. La verdadera maestría está en saber cuándo y cómo combinar diferentes enfoques."

La práctica profesional demuestra que la combinación estratégica de múltiples técnicas de diseño de casos de prueba produce resultados significativamente superiores a cualquier técnica individual. Cada enfoque tiene sus fortalezas y debilidades específicas.



Ejemplo práctico de combinación: En un sistema de login, usar partición de equivalencia y valores límite (caja negra) para validar diferentes tipos de credenciales, mientras se aplican pruebas de caminos (caja blanca) para verificar la lógica de autenticación, cifrado y manejo de sesiones.

04: Creación práctica de casos de prueba

La creación efectiva de casos de prueba es tanto arte como ciencia. Requiere pensamiento sistemático, atención al detalle y comprensión profunda tanto de los requisitos como de las técnicas disponibles. Este proceso estructurado garantiza casos de prueba de alta calidad.



Identificar requisitos

Analizar exhaustivamente la documentación de requisitos, especificaciones funcionales y criterios de aceptación. Identificar todas las condiciones de prueba, incluyendo casos normales, excepcionales y de borde.



Diseñar casos detallados

Documentar cada caso de prueba con: identificador único, descripción clara, precondiciones necesarias, pasos de ejecución específicos, datos de entrada precisos, resultados esperados verificables y postcondiciones relevantes.



Seleccionar técnica adecuada

Evaluar el contexto del proyecto, complejidad del sistema y recursos disponibles. Elegir la técnica más apropiada: caja negra para validación funcional, caja blanca para verificación estructural, o combinación según necesidad.



Revisar y refinar

Realizar revisión por pares para identificar ambigüedades, omisiones o redundancias. Asegurar que los casos sean reproducibles, mantenibles y alineados con los objetivos de calidad del proyecto.

Pausa de Pregunta – Evaluación de Estrategia

Diseñada para verificar la comprensión de las técnicas de caja negra y blanca durante la sesión.

- **Escenario:** Tenemos una función con una **complejidad ciclomática de 5**.
- **Pregunta:** ¿Cuántos casos de prueba mínimos se necesitan para garantizar que todos los caminos independientes sean ejecutados al menos una vez?
 - **A)** 1 caso de prueba representativo.
 - **B)** 5 casos de prueba diferentes.
 - **C)** Depende únicamente de las entradas del usuario (caja negra).
- **Reflexión rápida:** ¿Por qué es fundamental combinar el análisis de valores límite con la partición de equivalencia para detectar errores de codificación como el uso incorrecto de operadores.

Ejemplo práctico: Partición de Equivalencia

Requisito del sistema El campo de entrada debe aceptar exclusivamente valores numéricos dentro del rango válido de 1 a 10 (inclusive). Cualquier valor fuera de este rango, o de tipo no numérico, debe ser rechazado con un mensaje de error adecuado.

Identificación de clases de equivalencia

- 1.Clase válida: Valores numéricos entre 1 y 10 (inclusive)
- 2.Clase inválida inferior: Valores numéricos menores que 1
- 3.Clase inválida superior: Valores numéricos mayores que 10
- 4.Clase inválida de tipo: Valores no numéricos (alfabéticos, símbolos, etc.)

Casos de prueba representativos

ID Caso	Valor enviado	Clase de equivalencia	Resultado esperado
TC-01	5	Válida (1–10)	Aceptado
TC-02	-3	Inválida inferior	Rechazado con mensaje de error
TC-03	15	Inválida superior	Rechazado con mensaje de error
TC-04	"abc"	Inválida de tipo	Rechazado con mensaje de error

Ejemplo práctico: Análisis de Valores Límite

Contexto del requisito El sistema debe aceptar únicamente valores numéricos dentro del rango válido de 1 a 10 (inclusive). Cualquier valor fuera de este rango debe ser rechazado con un mensaje de error apropiado.

Aplicación del análisis de valores límite Esta técnica complementa la partición de equivalencia al enfocarse en los puntos críticos donde es más probable que se manifiesten errores de implementación, especialmente en los extremos del rango permitido.

Casos de prueba críticos

Nota técnica Los valores 0, 1, 10 y 11 son especialmente relevantes, ya que permiten detectar errores comunes de codificación, como el uso incorrecto de operadores de comparación ($<$ en lugar de $\leq$, por ejemplo).

Caso	Valor	Descripción	Resultado esperado
1	0	Justo antes del límite inferior	Rechazado con mensaje de error
2	1	Límite inferior exacto	Aceptado
3	2	Dentro del rango válido	Aceptado
4	9	Próximo al límite superior	Aceptado
5	10	Límite superior exacto	Aceptado
6	11	Justo después del límite superior	Rechazado con mensaje de error

Ejemplo práctico: Tabla de Decisión para Validación de Formulario

Contexto del sistema Se evalúa un formulario de envío con dos campos obligatorios: nombre completo y correo electrónico. El botón de envío debe habilitarse únicamente cuando ambos campos estén correctamente completados y validados. Para cubrir todas las combinaciones lógicas posibles, se aplica una tabla de decisión que permite identificar los casos de prueba críticos.

Condiciones evaluadas

- C1:** El campo "Nombre completo" está lleno
- C2:** El campo "Correo electrónico" contiene un formato válido

Tabla de decisión

Observación técnica Esta tabla genera cuatro casos de prueba que cubren todas las combinaciones posibles entre los dos campos. Su aplicación garantiza una validación exhaustiva del comportamiento del formulario, evitando omisiones en escenarios críticos.

Regla	R1	R2	R3	R4
Nombre completo	No	No	Sí	Sí
Email válido	No	Sí	No	Sí
Botón habilitado	No	No	No	Sí
Permitir envío	No	No	No	Sí
Mensaje de error	Ambos campos	Falta nombre	Email inválido	—

Herramientas de Gestión de Casos de Prueba

Las plataformas modernas de gestión de pruebas optimizan significativamente la eficiencia y efectividad del proceso de testing. Estas herramientas permiten centralizar, estructurar y automatizar componentes críticos del ciclo de vida de las pruebas, desde la creación hasta la ejecución y el análisis de resultados.

Beneficios clave

- **Documentación centralizada** Consolidación de todos los casos de prueba en un repositorio único, con control de versiones y trazabilidad de cambios para mantener un historial completo.
- **Reutilización eficiente** Identificación y reaprovechamiento de casos de prueba existentes, lo que reduce duplicaciones y agiliza la construcción de nuevas suites de validación.
- **Reportes avanzados** Generación de métricas, tableros visuales y reportes ejecutivos que respaldan la toma de decisiones basada en evidencia.
- **Trazabilidad completa** Vinculación directa entre pruebas, requisitos, defectos y resultados de ejecución, proporcionando visibilidad integral del estado de calidad.
- **Automatización integrada** Conexión con frameworks de automatización para ejecutar pruebas de forma automática y reportar resultados en tiempo real.
- **Colaboración mejorada** Facilitación del trabajo en equipo mediante funciones de comentarios, notificaciones y asignación de tareas entre miembros del equipo.

Herramientas destacadas TestRail · Jira + Zephyr · Azure Test Plans · qTest · TestLink · PractiTest



05. Mejores Prácticas y Cierre

Errores comunes en diseño de casos de prueba

Incluso testers experimentados pueden caer en trampas comunes que comprometen la efectividad de sus casos de prueba. Reconocer estos errores es el primer paso para evitarlos y mantener la calidad del proceso de testing.

Ambigüedad en la descripción

Pasos imprecisos o resultados esperados vagos como "el sistema funciona correctamente" no permiten una ejecución consistente ni verificación objetiva. **Solución:** Usar lenguaje específico, medible y verificable.

Cobertura insuficiente

No considerar casos extremos, condiciones de error o escenarios negativos deja vulnerabilidades sin detectar. **Solución:** Aplicar múltiples técnicas y pensar en modos de fallo.

Redundancia excesiva

Crear múltiples casos de prueba que validan exactamente lo mismo desperdicia recursos y complica el mantenimiento. **Solución:** Revisar y consolidar casos duplicados.

Dependencias no documentadas

Casos que dependen de precondiciones específicas o datos particulares sin documentarlo generan fallos impredecibles. **Solución:** Especificar claramente todas las precondiciones.

Falta de mantenimiento

No actualizar casos de prueba cuando los requisitos cambian crea desincronización entre pruebas y funcionalidad real. **Solución:** Implementar proceso de revisión con cada cambio.

Casos demasiado complejos

Intentar validar múltiples funcionalidades en un solo caso dificulta identificar la causa raíz de fallos. **Solución:** Mantener casos atómicos y enfocados.

Resumen y mejores prácticas



Claridad y precisión

Documentar cada caso de prueba con lenguaje claro, inequívoco y verificable. Cada paso debe ser reproducible por cualquier miembro del equipo sin ambigüedades.



Selección inteligente

Elegir las técnicas de diseño más apropiadas según el contexto específico: complejidad del sistema, criticidad de la funcionalidad, recursos disponibles y plazos del proyecto.



Combinación estratégica

Aprovechar las fortalezas de múltiples técnicas para lograr máxima cobertura y eficiencia. No limitarse a un solo enfoque cuando la combinación ofrece mejores resultados.



Mantenimiento continuo

Establecer un proceso sistemático de revisión y actualización de casos de prueba ante cada cambio en requisitos, asegurando sincronización constante con la realidad del sistema.

Principios fundamentales para el éxito

1. Priorizar la calidad sobre la cantidad de casos de prueba
2. Mantener trazabilidad entre requisitos y casos de prueba
3. Documentar suposiciones y limitaciones explícitamente
4. Fomentar la revisión por pares para detectar omisiones
5. Usar herramientas apropiadas para gestión y automatización
6. Medir y mejorar continuamente la efectividad de las pruebas

Conclusión: El poder de las técnicas avanzadas

"La calidad del software no es un accidente. Es el resultado de la aplicación disciplinada de técnicas probadas, combinada con pensamiento crítico y mejora continua."

Los casos de prueba meticulosamente diseñados utilizando técnicas avanzadas representan la diferencia fundamental entre simplemente probar software y garantizar verdaderamente su calidad. Son la primera línea de defensa contra fallos críticos que podrían impactar a usuarios finales, dañar la reputación de la organización o generar costos significativos de corrección post-producción.

10x

Retorno de inversión

Detectar y corregir defectos en etapas tempranas cuesta hasta 10 veces menos que corregirlos en producción

40%

Reducción de tiempo

Técnicas avanzadas reducen el tiempo de testing hasta un 40% mientras mejoran la cobertura

95%

Confiabilidad aumentada

Sistemas con casos de prueba bien diseñados alcanzan niveles de confiabilidad superiores al 95%

La maestría en técnicas avanzadas de diseño de casos de prueba no es solo una habilidad técnica; es una inversión estratégica en la excelencia del software. Optimiza el uso de recursos, mejora la confiabilidad del producto final y, en última instancia, contribuye directamente al éxito de los proyectos y la satisfacción de los clientes.

Te invitamos a aplicar estas técnicas en tus próximos proyectos y experimentar personalmente cómo elevan la calidad y el éxito de tu trabajo en testing de software.

Inversión en Excelencia

Un llamado a la acción final para aplicar lo aprendido como una ventaja estratégica.

- **El Poder de los Datos:**
 - **ROI:** Corregir defectos tempranamente cuesta hasta **10 veces menos** que hacerlo en producción.
 - **Eficiencia:** El uso de técnicas avanzadas puede reducir el tiempo de testing en un **40%**.
- **Invitación a Trabajar:** "La calidad del software no es un accidente. Es el resultado de la aplicación disciplinada de técnicas probadas".
- **Próximo Paso:** Te invitamos a aplicar estas técnicas (tablas de decisión, análisis de flujo de datos y caminos básicos) en tu próximo proyecto para elevar la confiabilidad de tus sistemas por encima del 95%.

Bibliografía Técnica y Normativa

- **IEEE.** (s.f.). *IEEE 610: Standard Glossary of Software Engineering Terminology*¹. Define el caso de prueba como un conjunto estructurado de entradas, condiciones y resultados para validar requisitos².
- **McCabe, T. J.** (1976). *A Complexity Measure*. IEEE Transactions on Software Engineering³. Desarrolló la métrica de complejidad ciclomática para determinar el número mínimo de casos de prueba necesarios para la cobertura de caminos⁴.
- **Mc Mullin.** (s.f.). *Fundamentos de Casos de Prueba*⁵. Destaca los casos de prueba como la herramienta principal para reducir costos y tiempos en el ciclo de vida del desarrollo de software⁶.

Herramientas de Gestión Mencionadas

- **Azure Test Plans.**
- **Jira + Zephyr.**
- **PractiTest.**
- **qTest.**
- **TestLink.**
- **TestRail.**

Felicitaciones ¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación